

Python C API and Pybind交互

目录

- paddle 中 pybind 相关设计:

pybind 及 python c 连接

解答_C_ops如何解析

paddle 中 pybind 相关设计:

BindImperative 为老动态图子module

BindEager(&m) 为新动态图子module

m.def("", &x) 其中""中是对外暴露的函数接口, &是c++函数的地址, 这个语法将c++端的代码和python连接, 做数据类型等转换。

(原有使用pybind语法连接python api 和 c++ kernel的转换过程耗时久, 因此paddle使用pythonc直接映射:pymethodDef)

</>

C 收起 ^

```
1 PYBIND11_MODULE(core_avx, m) {
2     BindImperative(&m);
3     BindEager(&m);
4     BindEagerStringTensor(&m);
5     BindCudaStream(&m);
6     BindJit(&m);
7
8     paddle::platform::CpuTotalPhysicalMemory();
9     paddle::memory::allocation::UseAllocatorStrategyGFlag();
10    AssertStaticGraphAndDygraphGradMakerNoDiff();
11    m.doc() = "C++ core of PaddlePaddle";
12
13    // using framework in this function. Since it is inside a function, it
14    // will not cause namespace pollution.
15    using namespace paddle::framework; // NOLINT
16
17    BindException(&m);
18    m.def("set_num_threads", &platform::SetNumThreads);
```

```

18 m.def("disable_signal_handler", &DisableSignalHandler);
19 m.def("clear_gradients",
20     [](std::vector<std::shared_ptr<imperative::VarBase>> param_list,
21         bool set_to_zero) {
22         for (auto param : param_list) {
23             param->ClearGradient(set_to_zero);
24         }
25     });
26 }

```

//定义子module后, 对外python调用为core_avx.eager.

</>

C | 收起 ^

```

1 void BindEager(pybind11::module* module) {
2     auto m = module->def_submodule("eager");
3
4     auto heap_type = reinterpret_cast<PyHeapTypeObject*>(
5         PyType_Type.tp_alloc(&PyType_Type, 0));
6     heap_type->ht_name = ToPyObject("Tensor");
7     heap_type->ht_qualname = ToPyObject("Tensor");
8     auto type = &heap_type->ht_type;
9     type->tp_name = "Tensor";
10    type->tp_basicsize = sizeof(TensorObject);
11    type->tp_dealloc = (destructor)TensorDealloc;
12    type->tp_as_number = &number_methods;
13    type->tp_as_sequence = &sequence_methods;
14    type->tp_as_mapping = &mapping_methods;
15    type->tp_methods = variable_methods;
16    type->tp_getset = variable_properties;
17    type->tp_init = TensorInit;
18    type->tp_new = TensorNew;
19    type->tp_weaklistoffset = offsetof(TensorObject, weakrefs);
20    Py_INCREF(&PyBaseObject_Type);
21    type->tp_base = reinterpret_cast<PyTypeObject*>(&PyBaseObject_Type);
22    type->tp_flags |=
23        Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE | Py_TPFLAGS_HEAPTYPE;
24    #if PY_VERSION_HEX >= 0x03050000
25        type->tp_as_async = &heap_type->as_async;
26    #endif
27    p_tensor_type = type;
28
29    if (PyType_Ready(type) < 0) {

```

```

30     PADDLE_THROW(platform::errors::Fatal(
31         "Init Paddle error in BindEager(PyType_Ready)."));
32     return;
33 }
34
35 Py_INCREF(type);
36 if (PyModule_AddObject(m.ptr(), "Tensor",
37     reinterpret_cast<PyObject*>(type)) <
38     0) {
39     Py_DECREF(type);
40     Py_DECREF(m.ptr());
41     PADDLE_THROW(platform::errors::Fatal(
42         "Init Paddle error in BindEager(PyModule_AddObject)."));
43     return;
44 }
45 BindFunctions(m.ptr());
46 BindEagerPyLayer(m.ptr());
47 BindEagerOpFunctions(&m);
48 }

```

m.ptr()是pyobject实例，BindEagerOpFunctions()。创建子模块 ops,使用 PyModule_AddFunctions(module, variable_functions)添加函数。python调用c++kernel时使用 core.eager.ops.xxx 。

</>

C++ | 收起 ^

```

1 void BindEagerOpFunctions(pybind11::module *module) {
2     InitOpsAttrTypeMap();
3     auto m = module->def_submodule("ops");
4     if (PyModule_AddFunctions(m.ptr(), ExttestMethods) < 0) {
5         PADDLE_THROW(platform::errors::Fatal ("Add functions to
6         core.eager.ops failed!"));
7     }
8     if (PyModule_AddFunctions(m.ptr(), CustomEagerMethods) < 0) {
9         PADDLE_THROW(platform::errors::Fatal ("Add functions to
10        core.eager.ops failed!"));
11    }
12    BindFinalStateEagerOpFunctions(&m);
13 }

```

基于上述pybind处理，老动态图python端调用c++代码使用前缀core_avx.ops，新动态图使用 core_avx.eager.ops，而在python/paddle/_c_ops.py中定义两个函数用来转换新老动态图。

__all__中删除全部老动态图函数，加入新动态图函数。

</>

Python | 收起 ^

```
1 def switch_to_eager_ops():
2     global _already_switch_to_eager_
3     if not _already_switch_to_eager_:
4         for name in dir(core.ops):
5             del globals()[name]
6             __all__.remove(name)
7         for name in dir(core.eager.ops):
8             globals()[name] = getattr(core.eager.ops, name)
9             __all__.append(name)
10    _already_switch_to_eager_ = True
```

添加函数语法如下：第一个字符串为添加的python端函数接口名称，第二个字段是添加的c++接口名称，第三个字段是python函数参数类型：METH_VARARGS指 (a,b)这样的变量参数；METH_KEYWORDS指 (a=1,b=2)这样的有初值参数,还有无参数类型METH_NOARGS，第四个字段是输出函数介绍信息。此表必须以一个空记录结尾 {NULL,NULL,0,NULL}

</>

C++ | 收起 ^

```
1 static PyMethodDef EagerFinalStateMethods[] = {
2     {"final_state_atan2", (PyCFunction)(void(*)(void))
3     eager_final_state_api_atan2, METH_VARARGS | METH_KEYWORDS, "C++
4     interface function for atan2 in dygraph."},
5     ...
6     {NULL,NULL,0,NULL}
7 }
```

在全部代码完成后，将c++代码编译成core_avx.so的动态链接库，则在此模块下暴露的所有c++函数都可以通过import core_avx后在python端调用。

